

Project 1: Automated Machine Learning

Human-Centric Artificial Intelligence

General instructions about projects

- The projects can be done alone or in groups of up to 5 students. The higher the number of students, the higher the expectations from our side.
- All projects will be grouped in one single django project, as explained below. It must be sent to us in the form of a Git repository.
- We will ask for the information on your group in a few weeks on StudIP.

1 Structure of the projects and first steps in django

The projects are implemented in python, with the django framework. Since the course is about human-centric aspects of machine learning, the projects are articulated on the interaction with humans. Each project will consist in a small application that is meant to be used by users. The apps will be accessed from a launch page.

1.1 Importing and launching the project

The skeleton of the project can be found on the link: <https://github.com/ppaamm/HCAI-PBL>. To launch the project, you must run the server. To do so, move to the root directory of the project, containing the file `manage.py` and run the following command:

```
python manage.py runserver
```

Once the server is running, you can access the project from your web browser, on the address <http://127.0.0.1:8000/>.

1.2 Editing the home page

We will now create the home page, from which you will offer the possibility to navigate through the projects. This homepage will display the main information about your group, as well as a list of links to each project.

In django, such a page is called an *app*. The home app is already created for you, in the directory `home`. In a django app, we identify the following important files:

- `models.py`: Contains the *models*, i.e. the structures of the elements that will be displayed. For the home page, this file is empty.
- `urls.py`: Describes the urls used in this app. Its main component is a list `urlpatterns` which contains paths. A path maps a url, a display method in the `views.py` and the unique name of the path.
- `views.py`: Defines the methods used to generate the views. In our context, this page contains one single method that generates the view from a HTML file.
- The `template` directory contains the templates, in particular the HTML files loaded in `views.py`. **Note that the path for the HTML file is important! It must always be `templates/[APP NAME]/[FILE NAME].html`.**

If you check the HTML file, you will see that it is not pure HTML, but only a template that includes some elements of python code. In particular, you can see two for loops: one for the description of the group, and one for creating the links to all projects.

Task 1

Get familiar with this app logic and edit the view to have the names and matriculation numbers of all group members appear on the page. Do this modification from the python, and not in the HTML!

1.3 Remark: Styles

Since the overall app is a web application, the whole display is managed in HTML. The style can be edited using CSS files. A global CSS file is located in the `static` directory at the root of the app. It is shared by all projects. In your projects, you might need to define a project-specific style sheet in addition to the global one.

A project-specific CSS style sheet is available in the directory `static/home`. For your next projects, it is important to keep this same path (such as for the HTML file).

The course is not about UX design, so do not put too much effort into the style sheets! However, do not neglect it too much and try to have some reasonable style for your website.

2 First project: Supervised Learning Interface

The goal of project 1 is to develop an interface for a simple supervised learning setting. This interface makes it possible to upload a dataset, visualize it and train several machine learning models.

2.1 Creating the app

To create the app for project 1, move to the root directory of the project and run the following command:

```
python manage.py startapp project1
```

Executing this will create the structure for an empty django app called `project1`. (Of course you can use any name you want for this app, not necessarily `project1`!). In order to make this app usable, you must follow certain important steps:

- **Register the app in `settings.py`:** Open `settings.py` (found in the project directory) and add your app to the list of `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'project1',  
]
```

Be careful with the quotation marks! If you simply copy-paste from the PDF, the encoding might not be the correct one.

- **Create `urls.py` in your app:** Inside the newly created app folder, create a file named `urls.py` with the following content:

```
from django.urls import path  
from . import views  
  
app_name = 'project1'  
  
urlpatterns = [  
    path('', views.index, name='index'),  
]
```

By defining the app name, you set a namespace so you can refer to this app's URLs in templates as `{% url 'project1:index' %}`. The `urlpatterns` list maps URL patterns (like `/`) to views in the app (you can have different views in one single app). The path that is being added to the list maps the root URL of the app to the index view. You can create other url patterns, for instance `path('/index', views.index, name='index')`, etc.

- **Write a first view:** Edit `views.py` in your app folder and add the view of your choice! It can be a very basic view, for instance:

```
from django.http import HttpResponse  
  
def index(request):  
    return HttpResponse("Welcome to Project 1!")
```

or something more sophisticated, inspired for instance by what is done in the home project. For now, you can start simple, but your goal will be to develop a real app in the end.

- **Connect Your App to the Main URL configuration:** Open the project's main `urls.py` (found next to `settings.py`) and include your app's URLs:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('home/', include('home.urls')),
    path("admin/", admin.site.urls),
    path('project1/', include('project1.urls')), # Add this line
]
```

This last step makes the app accessible from the URL `/project1`. To this base URL, you can access all the URLs defined within the project URL. Since we mapped in the `urlpatterns` the empty path to the `index` view, the URL `/project1/` will load this view. If you defined another `index` path, then `/project1/index` will load this view as well. This is how you can access different views within a single app.

Task 2

Create the app for the supervised learning interface project, and add a link to it in the home page by adding the following line (adapted to your choice of namespaces) to the projects list: `{"name": "Project 1", "url_name": "project1:index"},`.

2.2 Useful tools

For this project (and for the next ones), you will need to import files and to show plots and images. Examples of such functionalities are provided in the `demos` app.

Regarding the plots, there is no direct way to show a matplotlib figure in django. The solution is to generate the figure, save it into the media and load the image. These steps are shown in the demo app. An alternative solution would be to use JavaScript plotting libraries like ChartJS (<https://www.chartjs.org>).

2.3 Goal of the project

The app you are expected to develop for this project offers two functionalities to the user:

- Data visualization
- Training a ML model

These two functionalities correspond to the most basic ML pipeline and to steps that you have probably already done yourselves when analyzing some data.

2.4 Data loading and visualization

We consider here some scalar data (i.e. points in $\mathbb{R}^d$), where all features have a name. The CSV is structured in such a way that the first row corresponds to the feature names, and the last column to the output. For instance, the corresponding CSV for the Iris dataset would start with:

```
SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species
5.1, 3.5, 1.4, 0.2, 1
4.9, 3.0, 1.4, 0.2, 1
...
```

The description features are then the sepal length and width, and the petal length and width. The label to predict is the species of the flower. Some datasets are provided with an id for each point: you can either assume that the provided datasets do not include such a column, or filter it out.

In the Iris dataset, the primary goal is to train a classifier, though a regression approach can also be considered. For classification, one common way to visualize the data is by creating scatter plots of two selected features, using color to represent the different classes. For regression, visualization typically involves a scatter plot of either two features or one feature against the target label. You can choose whether your app supports classification problems, regression problems, or both. If it supports both, you'll need to decide whether the user must explicitly specify the type of problem, or whether the app should automatically detect it.

Task 3

Implement a module where you can upload a CSV file, read it and visualize the data.

You are free to choose the design you want, the visualizations you want etc. The minimum expected from you is a working solution for the given tasks, but the more you will do, the higher your grade!

2.5 Model training

Once the data are loaded and visualized, the user will want to train a machine learning model for the prediction. As we saw during the lecture, this involves:

- Choosing a ML model;
- Splitting the dataset into training and testing sets;
- Training the model for several values of the hyperparameter;
- Evaluating the trained models using a specific score.

You will have to implement this pipeline in order to offer an end-to-end interface for supervised learning. For the machine learning part, you can use the `scikit-learn` library.

Task 4

Implement this whole pipeline in your app. Think of what the user should be in control of, and what should be done automatically (in particular, is the user in charge of the model? of the splitting? of the hyperparameters? of the choice of the score?)

If you decide to implement several learning algorithms, you might have to define django models for the algorithms and the variables.